UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University
NAAC GRADE A+

WorkSheet -7

Student Name: Arnab Lala **UID:** 25MCI10232

Branch: M.C.A(A.I & M.L) **Section/Group:** MAM-1 A

Semester: Sem -2 **Date of Performance:**31/03/2026

Subject Name: TECHINCAL SKILLS **Subject Code:** 25CAP-652

Aim: Implementation of jOINS in PostgreSQL

S/W Requirement: Oracle Database Express Edition and pgAdmin

Objectives:

1. To understand how relational tables are connected using different types of SQL joins.
2. To implement INNER, LEFT, RIGHT, and CROSS JOIN operations on multiple tables.
3. To analyze relationships between students, courses, and departments using SQL queries.
4. To identify unmatched records using LEFT and RIGHT JOIN techniques.
5. To explore many-to-many relationships through a bridge (junction) table.
6. To generate meaningful insights by combining data from multiple tables.

Practical / Experiment Steps:

Step 0 Setup (Creation of Tables)

```
CREATE TABLE Studentss (
   student_id INT PRIMARY KEY,
   student_name VARCHAR(50),
   department_id INT
);
INSERT INTO Studentss VALUES (1, 'Arjun', 101);
INSERT INTO Studentss VALUES (2, 'Neha', 102);
INSERT INTO Studentss VALUES (3, 'Kabir', 103);
INSERT INTO Studentss VALUES (4, 'Meera', NULL);
INSERT INTO Studentss VALUES (5, 'Rohan', 101);
select * from Studentss;
```

	student_id [PK] integer	student_name character varying (50)	department_id integer
1	1	Arjun	101
2	2	Neha	102
3	3	Kabir	103
4	4	Meera	[null]
5	5	Rohan	101

```
CREATE TABLE Courses (
   course_id INT PRIMARY KEY,
   course_name VARCHAR(50));
INSERT INTO Courses VALUES (201, 'Database Systems');
INSERT INTO Courses VALUES (202, 'Artificial Intelligence');
INSERT INTO Courses VALUES (203, 'Computer Networks');
INSERT INTO Courses VALUES (204, 'Cloud Computing');
```

	course_id [PK] integer	course_name character varying (50)
1	201	Database Systems
2	202	Artificial Intelligence
3	203	Computer Networks
4	204	Cloud Computing

```
CREATE TABLE Enrollments (
   student_id INT,
   course_id INT,
   PRIMARY KEY (student_id, course_id),
   FOREIGN KEY (student_id) REFERENCES Studentss(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
INSERT INTO Enrollments VALUES (1, 201);
INSERT INTO Enrollments VALUES (1, 202);
INSERT INTO Enrollments VALUES (2, 203);
INSERT INTO Enrollments VALUES (3, 201);
INSERT INTO Enrollments VALUES (3, 204);
INSERT INTO Enrollments VALUES (5, 202);
```

	student_id [PK] integer	course_id [PK] integer
1	1	201
2	1	202
3	2	203
4	3	201
5	3	204
6	5	202

```
CREATE TABLE Departmentss (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(50)
);
INSERT INTO Departmentss VALUES (101, 'Computer Engineering');
INSERT INTO Departmentss VALUES (102, 'Data Science');
INSERT INTO Departmentss VALUES (103, 'Electronics & Communication');
INSERT INTO Departmentss VALUES (104, 'Mechanical');
select * from Departmentss;
```

	department_id [PK] integer	department_name character varying (50)
1	101	Computer Engineering
2	102	Data Science
3	103	Electronics & Communicati...
4	104	Mechanical

Step 1. Write queries to list students with their enrolled courses (INNER JOIN).

```
select s.student_name , c.course_name
from Studentss s
inner join Enrollments e
on s.student_id = e.student_id
inner join Courses c
on c.course_id = e.course_id;
```

Output:

	student_name character varying (50)	course_name character varying (50)
1	Arjun	Database Systems
2	Arjun	Artificial Intelligence
3	Neha	Computer Networks
4	Kabir	Database Systems
5	Kabir	Cloud Computing
6	Rohan	Artificial Intelligence

Step 2. Find students not enrolled in any course (LEFT JOIN).

```
select s.student_name
from Studentss s
left join Enrollments e
on s.student_id = e.student_id
where e.course_id is Null;
```

Output:

	student_name character varying (50)
1	Meera

Step 3. Display all courses with or without enrolled students

```
select distinct c.course_name
from courses c
 join Enrollments e
on c.course_id= e.course_id;
```

Output:

	course_name character varying (50)
1	Database Systems
2	Cloud Computing
3	Computer Networks
4	Artificial Intelligence

Step 4. Show students with department info using SELF JOIN or multiple joins.

```
select s.student_name, d.department_id, d.department_name
from Studentss s
join Departmentss d
on s.department_id = d.department_id
```

Output:

	student_name character varying (50)	department_id integer	department_name character varying (50)
1	Arjun	101	Computer Engineering
2	Neha	102	Data Science
3	Kabir	103	Electronics & Communicati...
4	Rohan	101	Computer Engineering

Step 5. Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

Output:

1	1	Arjun	101	201	Database Systems
2	2	Neha	102	201	Database Systems
3	3	Kabir	103	201	Database Systems
4	4	Meera	[null]	201	Database Systems
5	5	Rohan	101	201	Database Systems
6	1	Arjun	101	202	Artificial Intelligence
7	2	Neha	102	202	Artificial Intelligence
8	3	Kabir	103	202	Artificial Intelligence
9	4	Meera	[null]	202	Artificial Intelligence
10	5	Rohan	101	202	Artificial Intelligence
11	1	Arjun	101	203	Computer Networks
12	2	Neha	102	203	Computer Networks
13	3	Kabir	103	203	Computer Networks
14	4	Meera	[null]	203	Computer Networks
15	5	Rohan	101	203	Computer Networks
16	1	Arjun	101	204	Cloud Computing
17	2	Neha	102	204	Cloud Computing
18	3	Kabir	103	204	Cloud Computing
19	4	Meera	[null]	204	Cloud Computing
20	5	Rohan	101	204	Cloud Computing

Outcomes:

1. Ability to write SQL queries using different types of joins effectively.
2. Understanding of how to manage and query relational data across multiple tables.
3. Skill in identifying relationships like one-to-many and many-to-many in databases.
4. Capability to retrieve both matching and non-matching records using appropriate joins.
5. Improved problem-solving skills in designing and querying structured database systems.